

Fundamentals of Digital Electronics

Computer Architecture (ME-5319)

Phung Thanh Huy

Faculty of Mechanical Engineering
Ho Chi Minh City University of Technology (HCMUT) - VNUHCM

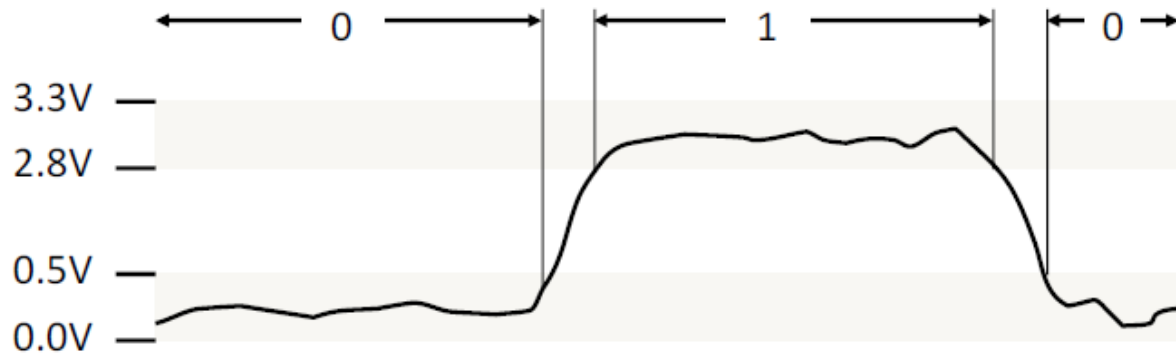
2026-v1



The Building Blocks of Modern Computers

Digital Abstraction

- In **digital electronic systems**, the **binary number system** is widely used to store and transmit data (**digital abstraction**).
- Binary is commonly used because it maps conveniently to **high and low voltage levels**, which correspond to logic values.
- Each binary value is called a **bit**.



- This forms the basis for representing data and performing operations in digital systems.

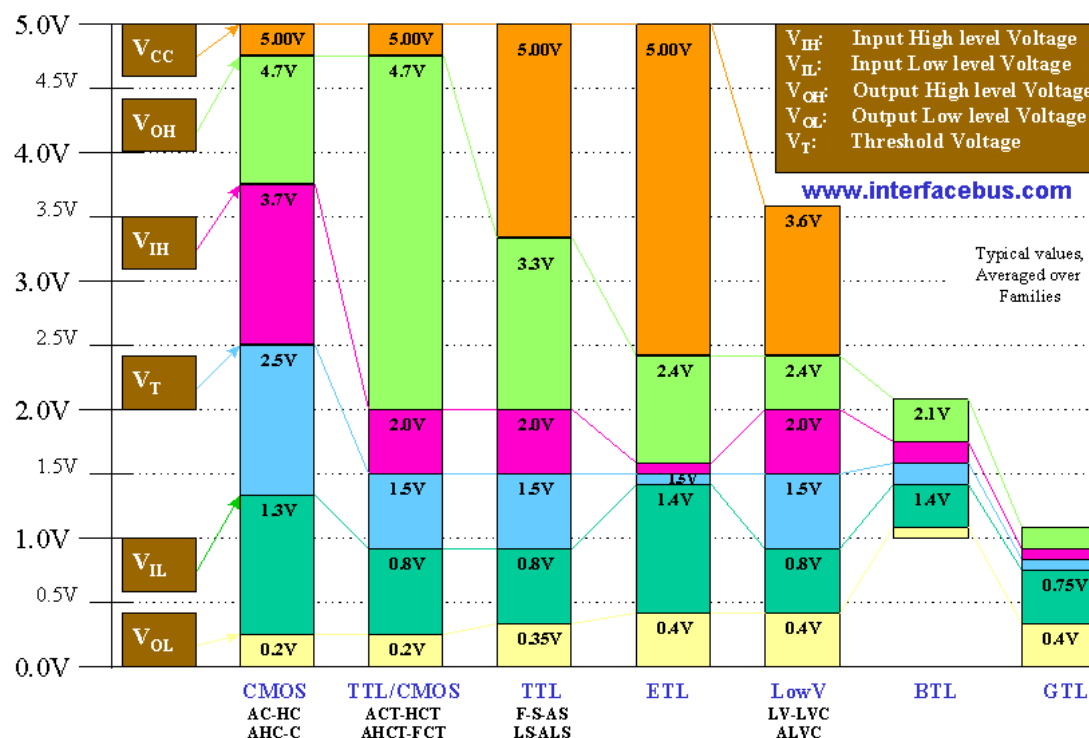
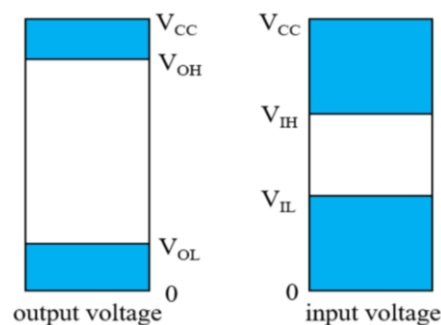
1 0 0 1
1 0 0 0 1 0 0 1

- When multiple bits are transmitted simultaneously, multiple **wires** are required, forming a **bus**.

The Building Blocks of Modern Computers

Digital Abstraction

- **Logic levels** represent the **ON/OFF states** of an electrical circuit and are defined according to specified voltage ranges.
- In **positive logic**, the **ON state** or **HIGH voltage level** is represented by **1**, while the **OFF state** or **LOW voltage level** is represented by **0**. In **negative logic**, the interpretation is reversed.
- **CMOS** and **TTL** are common logic-level standards used in digital electronic systems interfaces.

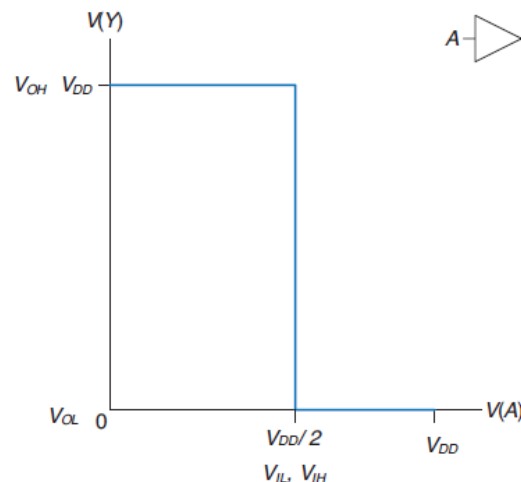
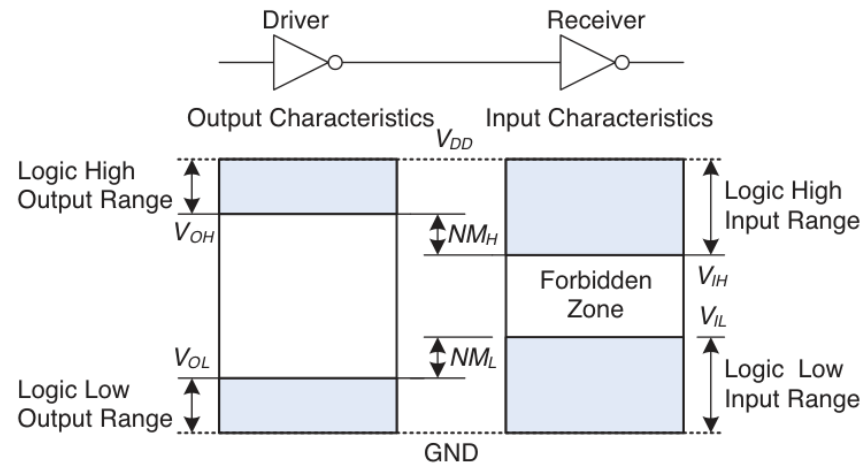


The Building Blocks of Modern Computers

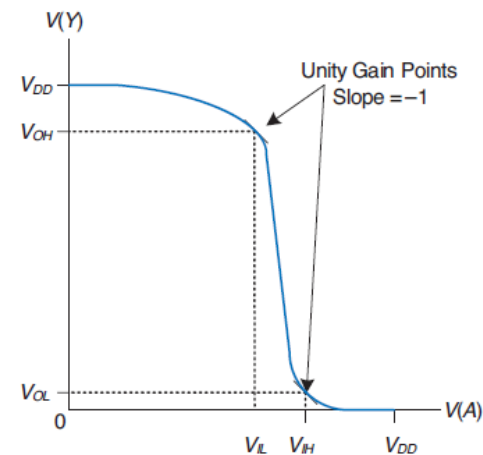
Digital Abstraction

$$NM_L = V_{IL} - V_{OL}$$

$$NM_H = V_{OH} - V_{IH}$$



(a) Ideal



(b) Practical

The Building Blocks of Modern Computers

Digital Abstraction

Logic Family	V_{DD}	V_{IL}	V_{IH}	V_{OL}	V_{OH}
TTL	5 (4.75–5.25)	0.8	2.0	0.4	2.4
CMOS	5 (4.5–6)	1.35	3.15	0.33	3.84
LVTTL	3.3 (3–3.6)	0.8	2.0	0.4	2.4
LVC MOS	3.3 (3–3.6)	0.9	1.8	0.36	2.7

Driver	Receiver			
	TTL	CMOS	LVTTL	LVC MOS
TTL	OK	NO: $V_{OH} < V_{IH}$	MAYBE ^a	MAYBE ^a
CMOS	OK	OK	MAYBE ^a	MAYBE ^a
LVTTL	OK	NO: $V_{OH} < V_{IH}$	OK	OK
LVC MOS	OK	NO: $V_{OH} < V_{IH}$	OK	OK

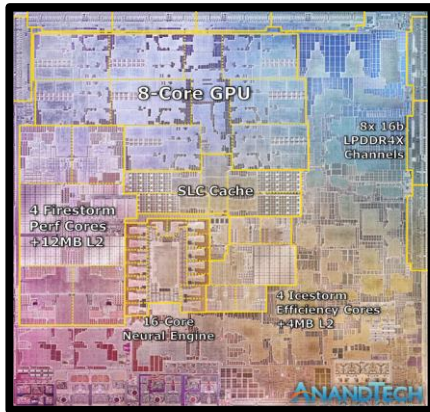
^aAs long as a 5 V HIGH level does not damage the receiver input

The Building Blocks of Modern Computers

General Purpose vs. Special Purpose Systems

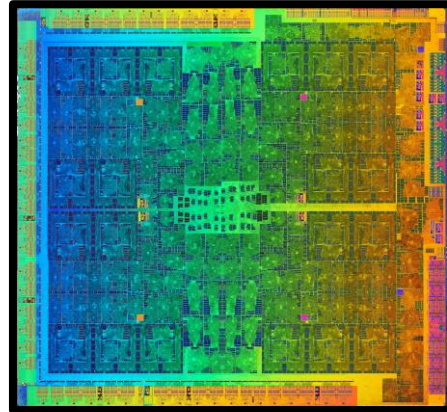
General Purpose

CPU_s



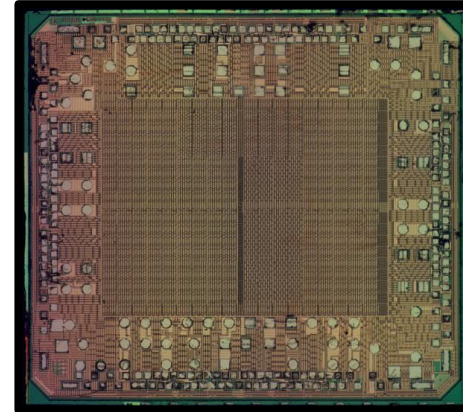
Apple M1

GPU_s



Nvidia GTX 1070

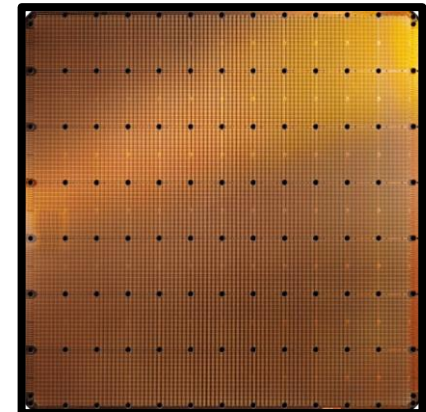
FPGA_s



Xilinx Spartan

Special Purpose

ASIC_s



Cerebras WSE-2



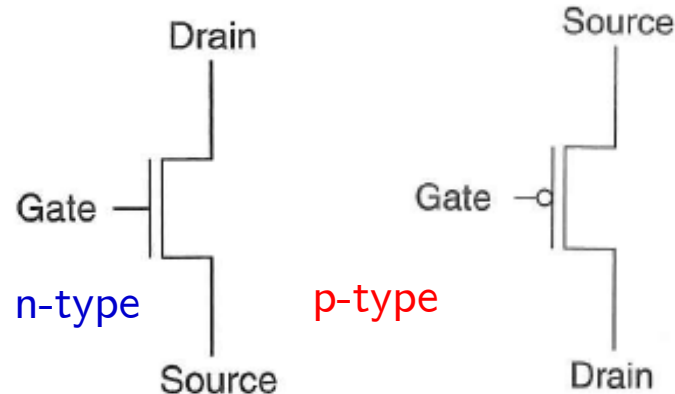
Flexible: Can execute any program
Easy to program & use

Efficient & High performance

The Building Blocks of Modern Computers

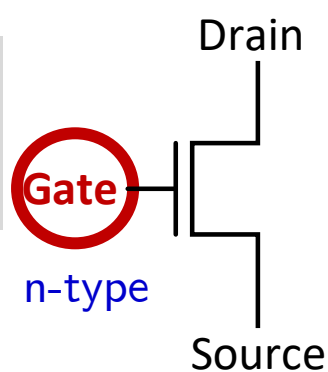
Transistors

- There are two types of MOS transistors: **n-type** and **p-type**

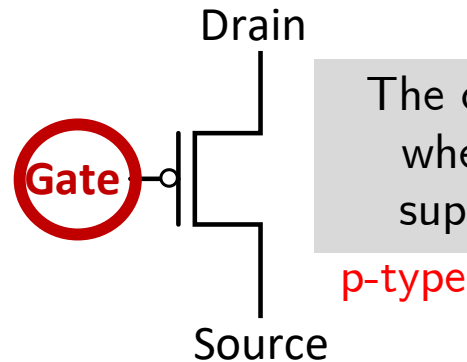


- They both operate “**logically**,” very similar to the way wall switches work

The circuit is closed when the gate is supplied with 3V

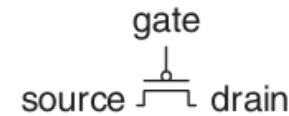
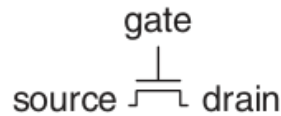
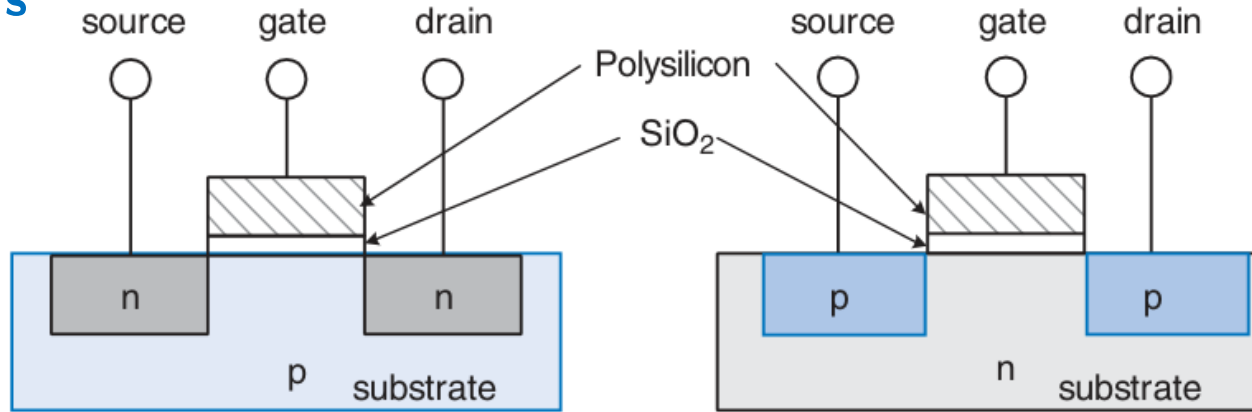


The circuit is closed when the gate is supplied with 0V

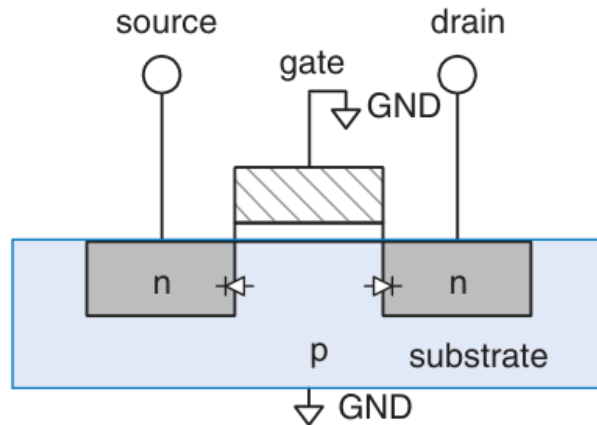


The Building Blocks of Modern Computers

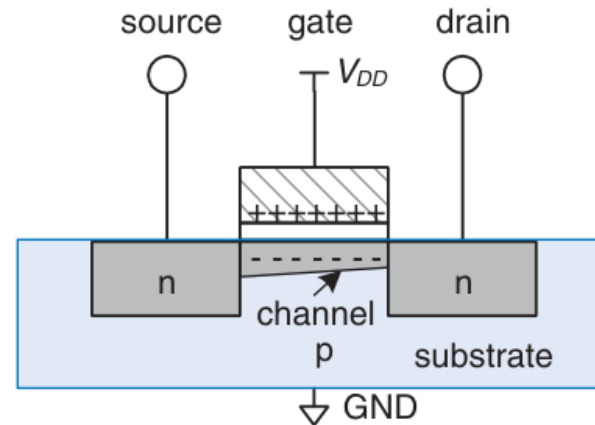
Transistors



nMOS



(a)



(b)

Number Systems

r-based numbers

□ The value of a number in a **base-r** system:

$$y_r = d_n r^n + d_{n-1} r^{n-1} + \dots + d_0 r^0 = \overline{d_n d_{n-1} \dots d_0}_r \equiv (\overline{d_n d_{n-1} \dots d_0})_r$$

- y_r : value of the number
- r : radix (or base).
- r^i is the weight
- d : digit.
- d_i is the digit number i^{th} .
- The value of $d_i \in \{0, 1, \dots, r - 1\}$.

□ Commonly used systems:

- Base 2: Binary. $d_i \in \{0, 1\}$
- Base 10: Decimal. $d_i \in \{0, 1, 2, \dots, 9\}$
- Base 8: Octal. $d_i \in \{0, 1, \dots, 7\}$
- Base 16: Hexadecimal. $d_i \in \{0, 1, 2, \dots, 9, A, B, C, D, E, F\}$

Number Systems

r-based numbers

- ❑ Extended for a fractional number.

$$(\overline{d_n \dots d_1 d_0} . \overline{d_{-1} \dots d_{-k}})_r = d_n r^n + \dots d_1 r + d_0 r^0 + d_{-1} r^{-1} + \dots + d_{-k} r^{-k}$$

- The dot to separate the positive and negative exponents is called radix point.
- Examples: decimal point, binary point, hexadecimal point,...

- ❑ Consider a integer number $y_r = (\overline{d_n d_{n-1} \dots d_0})_r$ and a fractional number

$$(\overline{d_n \dots d_1 d_0} . \overline{d_{-1} \dots d_{-k}})$$

- ❑ Most significant digit/bit (**MSB**): d_n (Left most).
- ❑ Least significant digit/bit (**LSB**): d_0 (integer) or d_{-k} (fractional) (Right most).
- ❑ Typically, decimal number system is used as a reference.

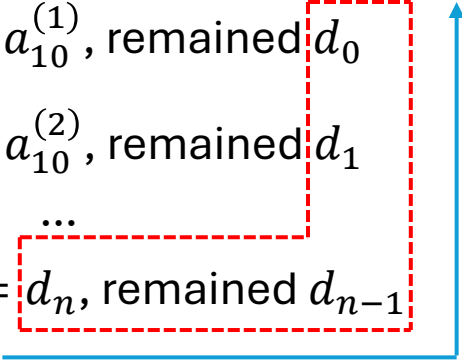
- ❑ Examples:

- $101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 5_{10}$
- $135_8 = 1 \times 8^2 + 3 \times 8^1 + 5 \times 8^0 = 93_{10}$
- $1FA_{16} = 1 \times 16^2 + 15 \times 16^1 + 10 \times 16^0 = 506_{10}$
- $10.01_2 = 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 2.25_{10}$

Number Systems

r-based numbers

❑ Conversion from a decimal number to base r number: **Repeated division method.**

$$a_{10} \leftrightarrow (\overline{d_n \dots d_1 d_0})_r$$
$$a_{10} \div r = a_{10}^{(1)}, \text{ remained } d_0$$
$$a_{10}^{(1)} \div r = a_{10}^{(2)}, \text{ remained } d_1$$
$$\dots$$
$$a_{10}^{(n-1)} \div r = d_n, \text{ remained } d_{n-1}$$


❑ Examples: Convert $(1000)_{10}$ to hexadecimal number.

Solution

$$(1000)_{10} \div 16 = (62)_{10}, \text{ remained } 8.$$

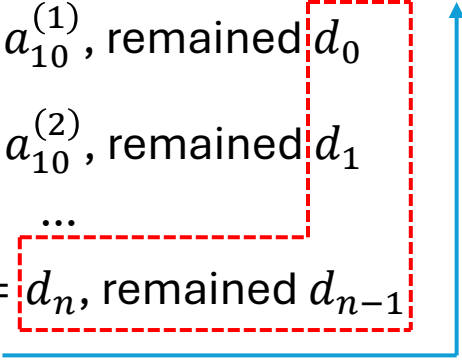
$$(62)_{10} \div 16 = 3, \text{ remained } 14 \text{ (E)}.$$

$$(1000)_{10} = (3E8)_{16}$$

Number Systems

r-based numbers

❑ Conversion from a decimal number to base r number: **Repeated division method.**

$$a_{10} \leftrightarrow (\overline{d_n \dots d_1 d_0})_r$$
$$a_{10} \div r = a_{10}^{(1)}, \text{ remained } d_0$$
$$a_{10}^{(1)} \div r = a_{10}^{(2)}, \text{ remained } d_1$$
$$\dots$$
$$a_{10}^{(n-1)} \div r = d_n, \text{ remained } d_{n-1}$$


❑ Examples: Convert $(100)_{10}$ to binary number.

Solution

$$(100)_{10} \div 2 = (50)_{10}, \text{ remained } 0. \quad (50)_{10} \div 2 = (25)_{10}, \text{ remained } 0.$$

$$(25)_{10} \div 2 = (12)_{10}, \text{ remained } 1. \quad (12)_{10} \div 2 = (6)_{10}, \text{ remained } 0.$$

$$(6)_{10} \div 2 = (3)_{10}, \text{ remained } 0. \quad (3)_{10} \div 2 = (1)_{10}, \text{ remained } 1.$$

$$(100)_{10} = (100010)_2$$

Number Systems

Binary number

❑ In Logical and Digital systems, Binary number system is commonly used to demonstrate the states. Examples:

- Positive Logic: True = 1; False = 0.
- Active high digital: Switched ON = 1; Switched OFF = 0.
- Active low digital: Switched ON = 0; Switched OFF = 1.

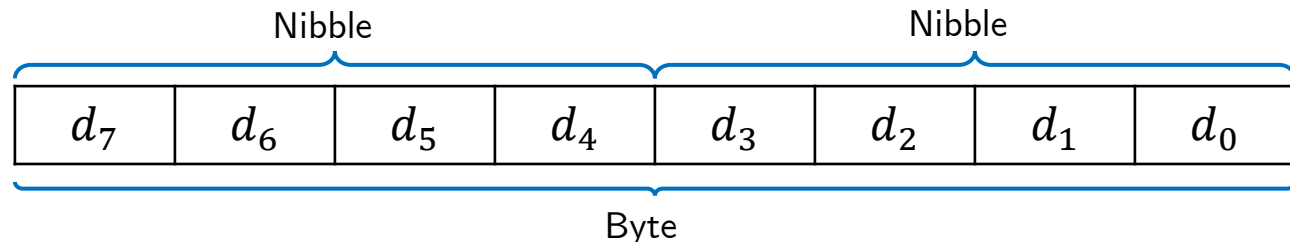
❑ Integer and Fractional Numbers:

$$y_2 = (\overline{d_n d_{n-1} \dots d_0})_2 = d_n \times 2^n + d_{n-1} \times 2^{n-1} + \dots + d_0 \times 2^0$$

$$y_2 = (\overline{d_n \dots d_1 d_0} \cdot \overline{d_{-1} \dots d_{-k}})_2$$
$$= d_n \times 2^n + \dots + d_1 \times 2^1 + d_0 + d_{-1} \times 2^{-1} + \dots + d_{-k} \times 2^{-k}$$

❑ Typically, binary numbers come with a **predefined number of digits**.

- Bit: 1 digit.
- Nibble: 4 bits.
- Byte: 8 bits.



Number Systems

Binary number

- ❑ In digital systems, large binary numbers with many digits are used. In such cases, octal numbers and hexadecimal numbers are usually used to reduced the length.
- ❑ Quick conversion **between binary number and octal number: 3-bit string** method

- Convert each group of 3 bits into octal digits. Concatenate the digits

Octal	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111

- Example:

$$(1101100)_2 = (001)_2 (101)_2 (100)_2 = (1)_8 (5)_8 (4)_8 = (154)_8$$

- ❑ Quick conversion **between binary number and hexadecimal number: 4-bit string** method

- Convert each group of 4 bits into hex digits. Concatenate the digits

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

- Example:

$$(1101100)_2 = (0110)_2 (1100)_2 = (6)_{16} (C)_{16} = (6C)_{16}$$

Number Systems

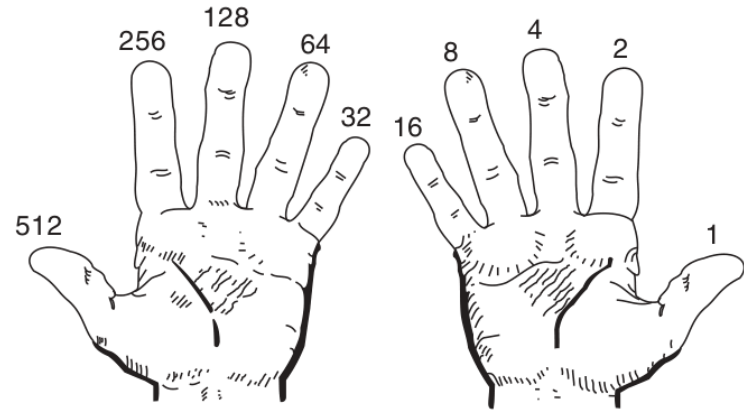
Binary number



Raised



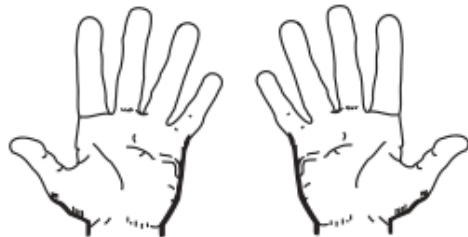
Lowered



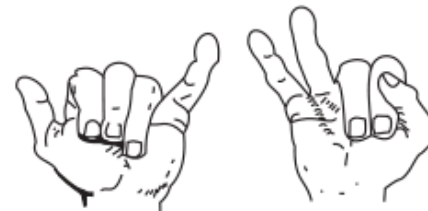
$$= 0000000000 = 0$$



$$= 0000010101 = 21$$



$$= 1111111111 = 1,023$$



$$= 1000111000 = 568$$

Number Systems

Binary number

❑ Binary addition

- General rule

$$\left\{ \begin{array}{l} 0 + 0 = 0 \\ 0 + 1 = 1 \\ 1 + 0 = 1 \\ 1 + 1 = 10_2 \end{array} \right\} \quad \left\{ \begin{array}{ll} 1 + 0 + 0 = 01_2 & 1 \text{ with a carry of } 0 \\ 1 + 0 + 1 = 10_2 & 0 \text{ with a carry of } 1 \\ 1 + 1 + 0 = 10_2 & 0 \text{ with a carry of } 1 \\ 1 + 1 + 1 = 11_2 & 1 \text{ with a carry of } 1 \end{array} \right\}$$

- In the case of 1+1, the value will be 0, then in the next step, 1 is added.

- Example:

(a)

$$\begin{array}{r} 1 \ 1 \ 1_2 \\ + 1 \ 1_2 \\ \hline 10 \ 1 \ 0_2 \end{array}$$

(b)

$$\begin{array}{r} 1 \ 1 \ 1 \ 0 \ 0_2 \\ + 1 \ 0 \ 0 \ 1 \ 1_2 \\ \hline 10 \ 1 \ 1 \ 1 \ 1_2 \end{array}$$

(c)

$$\begin{array}{r} 0 \ 1 \ 1 \ 0 \ 1_2 \\ + 0 \ 1 \ 1 \ 1 \ 0_2 \\ \hline 1 \ 1 \ 0 \ 1 \ 1_2 \end{array}$$

(d)

$$\begin{array}{r} 1 \ 1 \ 1 \ 1_2 \\ + 1 \ 1 \ 0 \ 0_2 \\ \hline 1 \ 1 \ 0 \ 1 \ 1_2 \end{array}$$

= 10, write 0, carry 1
= 10, add 1 (carried) = 11
write 1, carry 1
= 0, add 1 (carried) = 1

Number Systems

Binary number

❑ Binary subtraction

- General rule

$$\left\{ \begin{array}{l} 0 - 0 = 0 \\ 1 - 0 = 1 \\ 1 - 1 = 0 \\ 10_2 - 1 = 1 \end{array} \right\}$$

- In the case of $0 - 1$, borrow 1 from the next digit, then in the next step, 1 is subtracted.
- Example:

$$\begin{array}{r} 101_2 \\ 011_2 \\ \hline 10_2 \end{array}$$

Borrow 1, $10 - 1 = 1$.

$1 - 0 = 1$, $1 - 1$ (borrowed) = 0.

Number Systems

Binary number

❑ Binary multiplication

- General rule

$$\left\{ \begin{array}{l} 0 \times 0 = 0 \\ 0 \times 1 = 0 \\ 1 \times 0 = 0 \\ 1 \times 1 = 1 \end{array} \right\}$$

- Examples

(a)

$$\begin{array}{r} 11_2 \\ \times 10_2 \\ \hline 11_2 \\ 00_2 \\ \hline 100_2 \end{array}$$

(b)

$$\begin{array}{r} 1010_2 \\ \times 101_2 \\ \hline 1010_2 \\ 0000_2 \\ 1010_2 \\ \hline 10101_2 \end{array}$$

(c)

$$\begin{array}{r} 101_2 \\ \times 100_2 \\ \hline 101_2 \\ 000_2 \\ 000_2 \\ \hline 10100_2 \end{array}$$

❑ Binary Division

- Similar to Decimal Number

Number Systems

Binary number

❑ Overflow

- A number usually come with a fixed-length. For example, 8-bit number has 8 digits (bits), 4-bit number has 4 digits (bits), 16-bit number has 16 digits (bits),...
- If an operation produces a result that exceeds the range of the number system, overflow is said to occur.
- Example: Addition of 2 4-bit numbers:

$$(1101)_2 + (1011)_2 = (11000)_2 \rightarrow (1000)_2$$

The result has 5 bits. The MSB should be removed to remain 4-bit number. The final result should be $(1000)_2$. **This is a wrong result.**

- The overflow should be considered carefully in digital system.

Number Systems

Binary number

- ❑ Complement (Số bù) of a base r number system
 - Diminished radix complement: Each digit is substrated by $(r - 1)$
 - Radix complement: Diminished radix complement $+ 1$.
- ❑ Complement number is used to represent the signed (negative) number or quickly calculate the subtraction.

Hệ cơ số (Base)	Bù cơ số giảm (Diminished)	Bù cơ số (Radix)	Tính nhanh bù cơ số giảm	Công thức chuyển sang bù cơ số đủ
Nhị phân (Binary - Cơ số 2)	1's complement (Bù 1)	2's complement (Bù 2)	Đổi ngược toàn bộ bit: 0 thành 1 và 1 thành 0.	Lấy kết quả bù cơ số giảm cộng thêm 1.
Thập phân (Decimal - Cơ số 10)	9's complement (Bù 9)	10's complement (Bù 10)	Lấy 9 trừ đi từng chữ số của số đó.	Lấy kết quả bù cơ số giảm cộng thêm 1.
Thập lục phân (Hex - Cơ số 16)	15's complement (Bù 15)	16's complement (Bù 16)	Lấy F (15) trừ đi từng chữ số của số đó.	Lấy kết quả bù cơ số giảm cộng thêm 1.
Bát phân (Octal - Cơ số 8)	7's complement (Bù 7)	8's complement (Bù 8)	Lấy 7 trừ đi từng chữ số của số đó.	Lấy kết quả bù cơ số giảm cộng thêm 1.

Number Systems

Binary number

❑ Example 1: Consider number 90159_{10}

- 9's complement: $(9 - 9)(9 - 0)(9 - 1)(9 - 5)(9 - 9) = 09840$
- 10's complement: $09840 + 1 = 09841$

❑ Example 2: Calculate $456_{10} - 123_{10}$

Solution

- 9's complement: 876. 10's complement: 877
- $456 + 877 = 1333$.
- Remove "overflow digit" (digit 1)
- Results: 333.

❑ Example 3: Consider number 10010010_2

Solution

- 1's complement 01101101
- 2's complement $01101101 + 1 = 01101110$

Number Systems

Binary number

❑ Example 4: Consider number 11011011_2

- 1's complement 00100100
- 2's complement $00100100 + 1 = 00100101$

❑ Example 5: Calculate $11011011_2 - 10010010_2$

- 1's complement of 10010010 is 01101101
- 2's complement $01101101 + 1 = 01101110$
- $11011011_2 + 01101110_2 = 111001001$
- Remove the overflow digit, the result is 11001001_2

❑ Example 6: Calculate $AF10_{16} - 167C_{16}$

- 15's complement of $167C_{16}$ is $E983_{16}$
- 16's complement $E983 + 1 = E984$
- $AF10_{16} + E984_{16} = 1A994_{16}$
- Remove the overflow digit, the result is $A994_{16}$

Number Systems

Signed Binary Number

- ❑ Signed-Magnitude (Dấu-lượng): For a n-bit binary number
 - The Left most bit (LSB) is the signed bit. LSB = 1: Negative number. LSB = 0: Positive number.
 - Other (n-1) bits: Magnitude.
 - Examples: Convert into 8-bit binary number
 - $10_{10} = 00001010_2$
 - $-10_{10} = 10001010_2$
- ❑ 1's complement:
 - The negative number is the 1's complement of the positive number.
- ❑ 2's complement:
 - The negative number is the 2's complement of the positive number.
- ❑ 2's complement is mostly used (industrial standard) to prevent the problem of zero representation.

Number Systems

Signed Binary Number

□ Examples:

Decimal	Sign-Magnitude	1's Complement	2's Complement	
+5	0000 0101	0000 0101	0000 0101	Số dương giống nhau ở cả 3 cách.
+1	0000 0001	0000 0001	0000 0001	Bit dấu luôn là 0.
+0	0000 0000	0000 0000	0000 0000	Số 0 dương.
-0	1000 0000	1111 1111	Không tồn tại	Bù 2 loại bỏ số -0 thừa.
-1	1000 0001	1111 1110	1111 1111	
-5	1000 0101	1111 1010	1111 1011	

Number Systems

Floating Point Number

- ❑ Digital systems usually use Floating Point Format to handle real/ fractional numbers.
- ❑ Many hardware floating-point units use the IEEE 754 standard (IEC-60559).
- ❑ A fractional number x_r can be represented in binary format by:

$$x_r = (-1)^S \times 1.M \times 2^{(E-Bias)}$$

Sign (1 bit)	Exponent	Mantissa (Fraction)
--------------	----------	---------------------

Types in IEEE 754	Name	Total bit	Sign bit	Exponent bit	Mantissa bit	Bias
binary16	Half precision	16 bit	1 bit	5 bit	10 bit	$(2^{5-1} - 1) = 15$
binary32	Single precision (Float/ Single)	32 bit	1 bit	8 bit	23 bit	$(2^{8-1} - 1) = 127$
binary64	Double precision (Double)	64 bit	1 bit	11 bit	52 bit	$(2^{11-1} - 1) = 1023$
binary128	Quadruple precision	128 bit	1 bit	15 bit	112 bit	$(2^{15-1} - 1) = 16383$

Number Systems

Floating Point Number

□ Example

a) Present the number 9.625 in 32-bit FP

Solution:

- $S = 0$ (Positive)
- Fixed point representation: $9.625_{10} = 1001.101_2$
- Shift the point 3 digit: $1001.101_2 = 1.001101_2 \times 2^3$
- Mantissa: **001101**00000000000000000000
- $(E - Bias) = 3$. Bias = 127 (32-bit) $\rightarrow E = 130_{10} = 10000010_2$

0	10000010	0011010000000000000000000000
---	----------	------------------------------

Number Systems

Floating Point Number

□ Example

b) Present the number -0.75 in 32-bit FP

Solution:

- $S = 1$ (Negative)
- Fixed point representation: $0.75_{10} = 0.11_2$
- Shift the point 1 digit: $0.11_2 = 1.\textcolor{red}{1}_2 \times 2^{-1}$
- Mantissa: $\textcolor{red}{1}000000000000000000000000$
- $(E - \textit{Bias}) = -1$. Bias = 127 (32-bit) $\rightarrow E = 126_{10} = 01111110_2$

1	01111110	100000000000000000000000
---	----------	--------------------------

Number Systems

Floating Point Number

❑ Example

c) Find the value of the number 0 01111100 100000000000000000000000

Solution:

- $S = 0$ (Positive)
- $E = 01111100_2 = 124. \rightarrow E - bias = 124 - 127 = -3$
- Mantissa: 100000000000000000000000 $\rightarrow 1.M = 1.1_2 = 1.5_{10}$
- $Value = +1.5 \times 2^{-3} = 0.1875$

❑ Notes: Floating point representation can produce some errors.

- Rounding Error
- Accumulation Error
- Loss of Significance / Cancellation
- Overflow and Underflow

❑ Moreover, using Floating Point requires more computation than integers.

Number Systems

Floating Point Number

❑ Some special values

Special Value	Sign Bit (S)	Exponent Bits (E)	Fraction/Mantissa Bits (M)	Meaning and Occurrence
Positive Zero (+0)	0	All 0s	All 0s	Represents the default value of zero.
Negative Zero (-0)	1	All 0s	All 0s	Occurs when a negative number is divided by ∞ , or when a negative calculation results in an underflow.
Subnormal / Denormalized	0 or 1	All 0s	Non-zero (At least one bit is 1)	Used to represent extremely small numbers close to 0 to prevent sudden underflow. The formula bypasses the implicit leading 1: $x_r = (-1)^S \times 0.M \times 2^{1-Bias}$
Positive Infinity ($+\infty$)	0	All 1s	All 0s	Occurs when a positive number is divided by 0, or when a positive calculation results in an overflow.
Negative Infinity ($-\infty$)	1	All 1s	All 0s	Occurs when a negative number is divided by 0, or when a negative calculation results in an overflow.
Not a Number (NaN)	0 or 1	All 1s	Non-zero (At least one bit is 1)	Represents an invalid or undefined mathematical operation. Examples: $\frac{0}{0}, \infty - \infty, \sqrt{-1}$

Number Systems

Floating Point Number

❑ Floating point addition

Example Calculate $(1.1101_2 \times 2^3 + 1.0100_2 \times 2^1)$

- Step 1: Compare and Align Exponents
 - Find the number with the smaller exponent. Here, 2^1 is smaller than 2^3 .
 - Shift its decimal point to the left by the difference ($3 - 1 = 2$ places) and increase its exponent.
 - $(1.0100_2 \times 2^1 \rightarrow 0.10100_2 \times 2^2 \rightarrow \mathbf{0.0101_2 \times 2^3})$
- Step 2: Add the Mantissas
 - Add the two values like standard integers, keeping the decimal aligned: $1.1101_2 + 0.0101_2 = 10.0010_2$
- Step 3: Normalize the result.
 - Result: $10.0010 \times 2^3 = 1.00010 \times 2^4$
 - New $E - bias = 4$
 - New Mantissa = 00010

Number Systems

Floating Point Number

❑ Floating point subtraction

Example Calculate $(1.0000_2 \times 2^2 + 1.1000_2 \times 2^0)$

- Step 1: Compare and Align Exponents
 - Find the number with the smaller exponent. Here, 2^0 is smaller than 2^2 .
 - Shift its decimal point to the left by the difference ($2 - 0 = 2$ places) and increase its exponent.
 - $(1.1000_2 \times 2^0 \rightarrow 0.0110_2 \times 2^2)$
- Step 2: Subtract the Mantissas
 - Add the two values like standard integers, keeping the decimal aligned:
 - $1.0000_2 + 0.0110_2 = 0.1010_2$
- Step 3: Normalize the result.
 - Result:
 - $0.1010 \times 2^2 = 1.0100 \times 2^1$
 - New $E - bias = 1$
 - New Mantissa =

0010

Number Systems

Floating Point Number

❑ Floating point multiplication

Example Calculate $(1.1100_2 \times 2^3) \times (1.0100_2 \times 2^2)$

☒ Step 1: Add exponent

$$2^3 \times 2^2 = 2^5.$$

- Step 2: Multiply the mantissas

- Multiply without the point.

$$1.1100_2 \rightarrow 11100_2 \times 2^4$$

$$1.0100_2 \rightarrow 10100_2 \times 2^4$$

$$11100 \times 10100 = 1000110000$$

- Add the point

$$1000110000 \rightarrow 1000110000 \times 2^{-8} = 10.00110000$$

- Step 3: Normalize the result.

- Result:

$$10.00110000 \times 2^5 = 1.000110000 \times 2^6$$

- New $E - bias = 6$
- New Mantissa

000110000

Codes

Code, Encode and Decode

- ❑ When numbers, letters, or words are represented by a special group of symbols they are **encoded**.
- ❑ The group of symbols is called a code.
- ❑ For more specific in a digital system, a particular combination of n bit-values is called a code word.
- ❑ **Straight binary coding**: a decimal number is represented by its equivalent binary number.
- ❑ Digital systems use binary numbers, while the real world uses decimal.
- ❑ Frequent conversions between the two are necessary but become complex for large numbers.
- ❑ Hybrid encoding that combines features of both decimal and binary systems is used to simplify this process.
- ❑ Special codes are also utilized for auxiliary functions, such as error detection.

Codes

BCD (Binary-Coded Decimal):

- ❑ Represent the decimal digits (0-9) by 4-bit binary number.
- ❑ BCD is a weighted binary code, i.e., the position of the bit has significant weight.

$$d_{BCD} = d_3w_3 + d_2w_2 + d_1w_1 + d_0w_0$$

(w_3, w_2, w_1, w_0) are weights.

- ❑ Original BCD is 8421 (i.e., $2^3, 2^2, 2^1, 2^0$).

$$d_{BCD} = d_3 \times 8 + d_2 \times 4 + d_1 \times 2 + d_0$$

- ❑ Another popular BCD is 2421, which help the subtraction more efficient.

$$d_{2421} = d_3 \times 2 + d_2 \times 4 + d_1 \times 2 + d_0$$

Digit	0	1	2	3	4	5	6	7	8	9
8421 BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
2421 BCD	0000	0001	0010	0011	0100	0101	0110	1101	1110	1111
5421 BCD	0000	0001	0010	0011	0100	1000	1001	1010	1011	1100

BCD (Binary-Coded Decimal):

- ❑ Examples: Represent the following numbers in BCD and BCD-2421 codes.

$$562_{10}, 9.2_{10}, 321_{10}$$

Solution. Map the table:

$$562_{10} \rightarrow 0101\ 0110\ 0010 \text{ (both 8421 – 2421)}$$

$$9.2 \rightarrow 1001.0010 \text{ (8421)}$$

$$9.2 \rightarrow 1111.0010 \text{ (2421)}$$

$$321_{10} \rightarrow 0011\ 0010\ 0001 \text{ (8421 – 2421)}$$

- ❑ There are other weights for BCD. For example, **5421-BCD**, for **digital counting** applications.
- ❑ The main advantage of the BCD code is the relative ease of converting to and from decimal. Also, BCD keep the whole information from the decimal numbers.
- ❑ This ease of conversion is especially important from a hardware standpoint because in a digital system, the logic circuits perform conversions to and from decimal.

Codes

BCD (Binary-Coded Decimal):

❑ BCD addition.

❑ Add 2 numbers using the rules for binary numbers.

❑ If the results < 9 , it is a valid result.

❑ If the results > 9 , add 6 (0110) and take 4-bit. Carry 1. Add carry to the next.

❑ Example: Calculate

a) 0110 0111 + 0101 0011

Solution

$0111 + 0011 = 1010 (= 10 > 9)$.

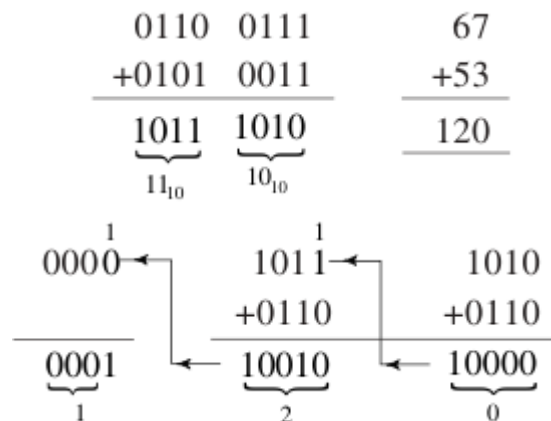
$1010 + 0110 = 10000 \rightarrow 0000$, carry 1.

$0110 + 0101 = 1011 (= 11 > 9)$.

Take $1011 + 0110 + \text{carry} = 10010 + 1 \rightarrow 10011 \rightarrow 0011$, carry 1.

$0000 + 0000 + \text{carry} = 0001$.

Result: 0001 0011 0000



Codes

BCD (Binary-Coded Decimal):

❑ Example. Calculate

0100 0100 1000 + 0100 1000 1001

Solution

1000 + 1001 = 10001 (>9)

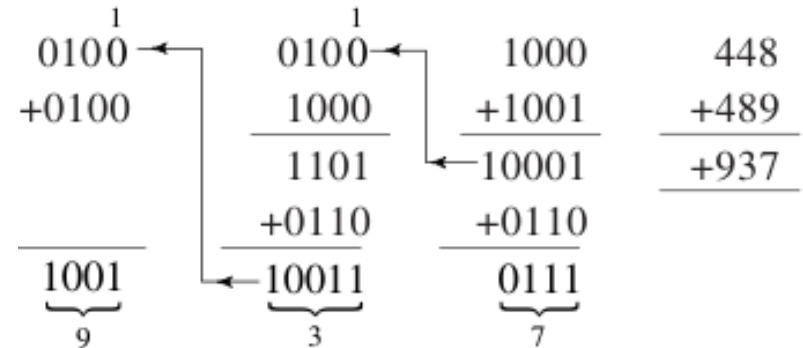
Take $10001 + 0110 = 10111 \rightarrow 0111$, carry 1.

0100 + 1000 + carry = 1101 (>9)

Take $1101 + 0110 = 10011 \rightarrow 0011$, carry 1.

0100 + 0100 + carry = 1001 (= 9, valid)

Result: 1001 0011 0111



Codes

EBCDIC (eb-si-dic): Extended Binary Coded Decimal Interchange Code

❑ Change the digits from 4-bit to 8-bit by add 1111

Digit	0	1	...	8	9
8421 BCD	0000	0001	...	1000	1001
EBCDIC	11110000	11110001	...	11111000	11111001

Excess Code

❑ Excess-K

- The digit is alternated by $\text{digit} + K$.
- Typical value of K is $2^{m-1} - 1$. For example, the Exponent E in IEEE 754 is coded by $E + 2^{m-1} - 1$, where m is the length of the exponent bit.

❑ Excess-3 Coding (Excess 3 – BCD, XS-3)

- The digit is first added 3
- The added digit is converted into binary.

Decimal digit	0	1	2	3	4	5	6	7	8	9
Add 3	3	4	5	6	7	8	9	10	11	12
Excess-3 Code	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

- Examples

$$(25)_{10} \rightarrow 0101\ 1000$$

$$(189)_{10} \rightarrow 0100\ 1011\ 1100$$

Codes

Excess Code

❑ Excess-3 Addition

- ❑ Step 1: Add the binary number
- ❑ If there is no carry out of the four-bit group, subtract 011
- ❑ If there is a carry out of four-bit group, add 011. In this case, if the 5th bit is 1, change it into 0100.

❑ Example. Calculate in Excess-3 Code: $2 + 3$

Solution

$$2 \rightarrow 0101;$$

$$3 \rightarrow 0110;$$

$$0101 + 0110 = 1011$$

There is no carry.

$$1011 - 0011 = 1000$$

The result is 1000.

Codes

Excess Code

❑ Example. Calculate in Excess-3 Code: $6 + 8$.

Solution

$$6 \rightarrow 1001;$$

$$8 \rightarrow 1011;$$

$$1001 + 1011 = 10100$$

There is a carry (bit 5):

$$10100 + 0011 = 10111$$

The 5th bit is 1, the result is

$$0100\ 0111$$

Excess Code

❑ Example. Calculate in Excess-3 Code: $27 + 35$

☒ First digit:

$$7 \rightarrow 1010;$$

$$5 \rightarrow 1000;$$

$$1010 + 1000 = 10010$$

There is a carry (bit 5): $10010 + 0011 = 10011$. Carry is 1 $\rightarrow 0100$

☒ Second digit: (need to add carry from the first digit)

$$2 \rightarrow 0101$$

$$3 \rightarrow 0110$$

$$0101 + 0110 + 0100 = 1100$$

There is no carry in the result.

$$1100 - 0011 = 1001$$

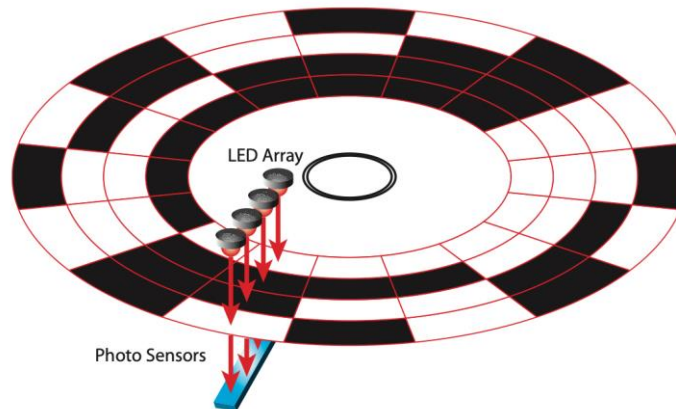
☒ Final result: 1001 0100

Codes

Gray Code

❑ Gray code

- ☒ The Gray code belongs to a class of codes called mini-mum change code (cyclic code).
- ☒ When the number increase 1 unit, only 1 bit in the code string changes.
- ☒ The Gray code is an unweighted code.
- ☒ Because of this the Gray code is not suitable for arithmetic operations but finds application in input/output devices, some types of analog-to-digital converters, and designation of rows and columns in a Karnaugh map,...



Codes

Gray Code

□ Gray code

- ⊗ The code can have as many bit as necessary.
- ⊗ A n-bit code can represent the number form $0 \rightarrow 2^n - 1$

	2-bit	3-bit	4-bit
0	0 0	0 0 0	0 0 0 0
	0 1	0 0 1	0 0 0 1
	1 1	0 1 1	0 0 1 1
3	1 0	0 1 0	0 0 1 0
		1 1 0	0 1 1 0
		1 1 1	0 1 1 1
		1 0 1	0 1 0 1
7		1 0 0	0 1 0 0
			1 1 0 0
			1 1 0 1
			1 1 1 1
			1 1 1 0
			1 0 1 0
			1 0 1 1
			1 0 0 1
15			1 0 0 0
(a)	(b)	(c)	

Codes

Alphanumeric Codes

☐ ASCII Code

☒ The ASCII Code: 7-bit (Stored as 8-bit, with $b_7=0$)

Letter A (Capitalized)

- Binary: $0100\ 0001_2$
- Hex: 41_{16}
- Decimal: 65_{10}

		$b_6b_5b_4$ (column)							
$b_3b_2b_1b_0$	Row (hex)	000 0	001 1	010 2	011 3	100 4	101 5	110 6	111 7
0000	0	NUL	DLE	SP	0	@	P	'	p
0001	1	SOH	DC1	!	1	A	Q	a	q
0010	2	STX	DC2	"	2	B	R	b	r
0011	3	ETX	DC3	#	3	C	S	c	s
0100	4	EOT	DC4	\$	4	D	T	d	t
0101	5	ENQ	NAK	%	5	E	U	e	u
0110	6	ACK	SYN	&	6	F	V	f	v
0111	7	BEL	ETB	,	7	G	W	g	w
1000	8	BS	CAN	(	8	H	X	h	x
1001	9	HT	EM	)	9	I	Y	i	y
1010	A	LF	SUB	*	:	J	Z	j	z
1011	B	VT	ESC	+	;	K	[	k	{
1100	C	FF	FS	,	<	L	\	l	
1101	D	CR	GS	-	=	M	]	m	}
1110	E	SO	RS	.	>	N	^	n	~
1111	F	SI	US	/	?	O	_	o	DEL

Codes

Alphanumeric Codes

❑ ASCII Code

- The ASCII Code: 7-bit (Stored as 8-bit, with $b_7=0$)
- There are some control characters/ codes.

Control codes

NUL	Null	DLE	Data link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronize
BEL	Bell	ETB	End transmitted block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete or rubout

- ❑ There are also Extended ASCII Codes, full 8-bit, 256 characters.
- ❑ Modern systems also use modern codes such as Unicode (UTF-8, UTF-32,...)

Codes

Codes for action, conditions, and states

- ❑ In digital system design, we often encounter non-data applications where a string of bits must be used to control an action, to flag a condition, or to represent the current state of the hardware.
- ❑ If there are n different actions, conditions, or states, we can represent them with a b -bit binary code with

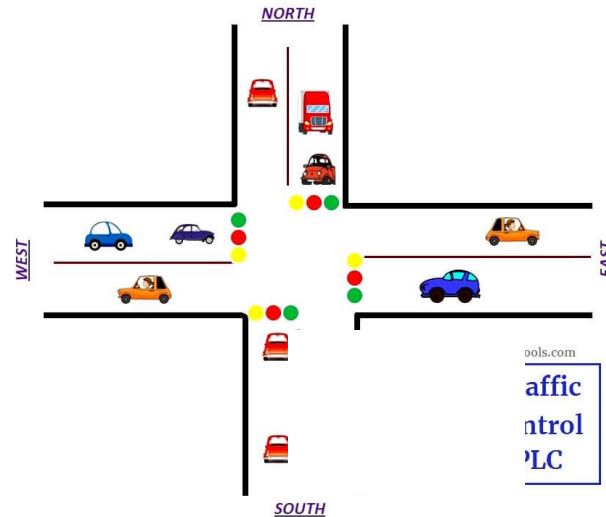
$$b = \lceil \log_2 n \rceil$$

Or $2^b \geq n$

Codes

Codes for action, conditions, and states

□ Example. States of a traffic light station.



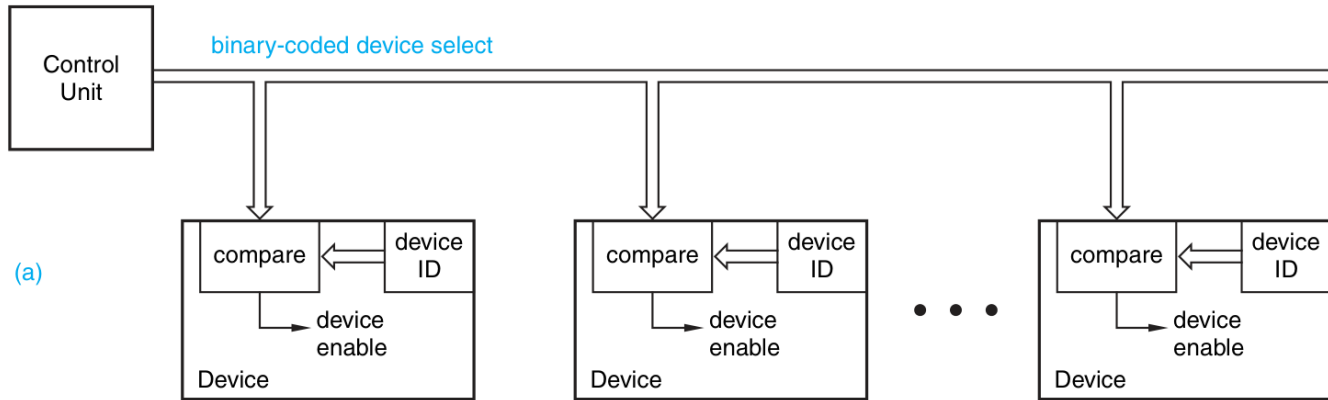
Lights

State	N-S Green	N-S Yellow	N-S Red	E-W Green	E-W Yellow	E-W Red	Code Word
N-S go	ON	off	off	off	off	ON	000
N-S wait	off	ON	off	off	off	ON	001
N-S delay	off	off	ON	off	off	ON	010
E-W go	off	off	ON	ON	off	off	100
E-W wait	off	off	ON	off	ON	off	101
E-W delay	off	off	ON	off	off	ON	110

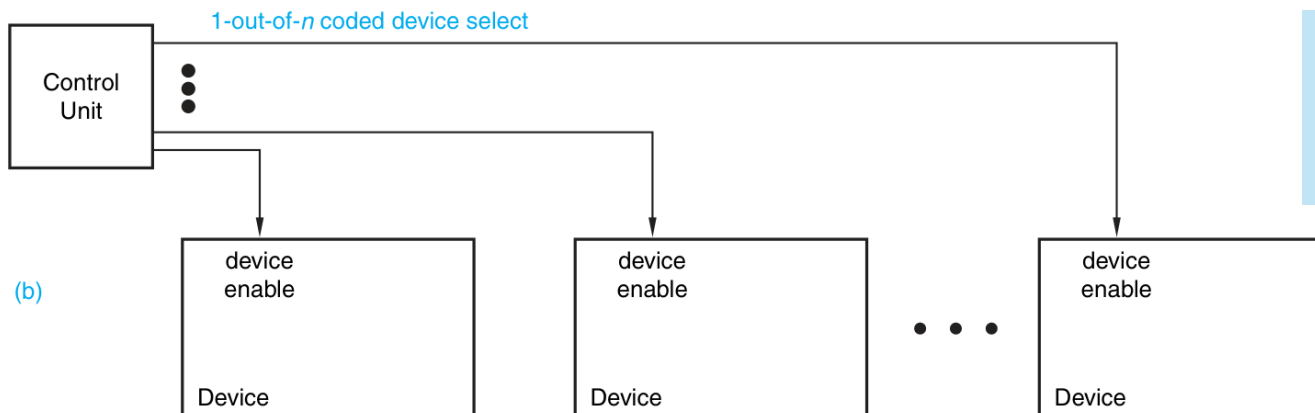
Codes

Codes for action, conditions, and states

❑ Example. Code to define the selected devices.



Device i :
Represent i in
binary.

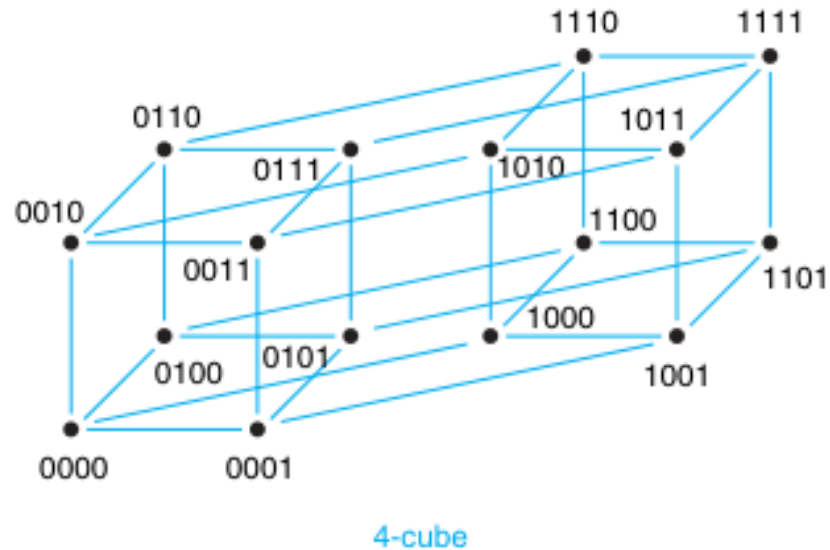
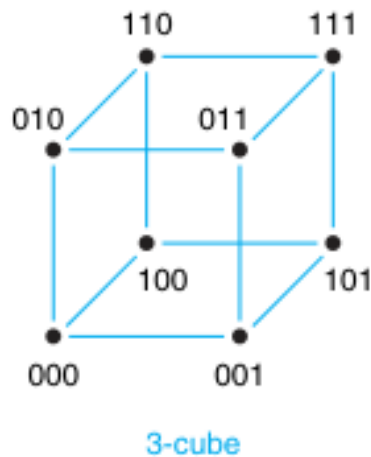
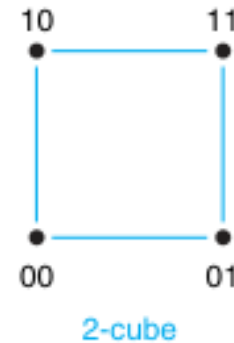


A number with n -bit.
If device i is selected,
the bit $(i - 1)^{th}$ is 1.

Codes

n-Cubes and Distance

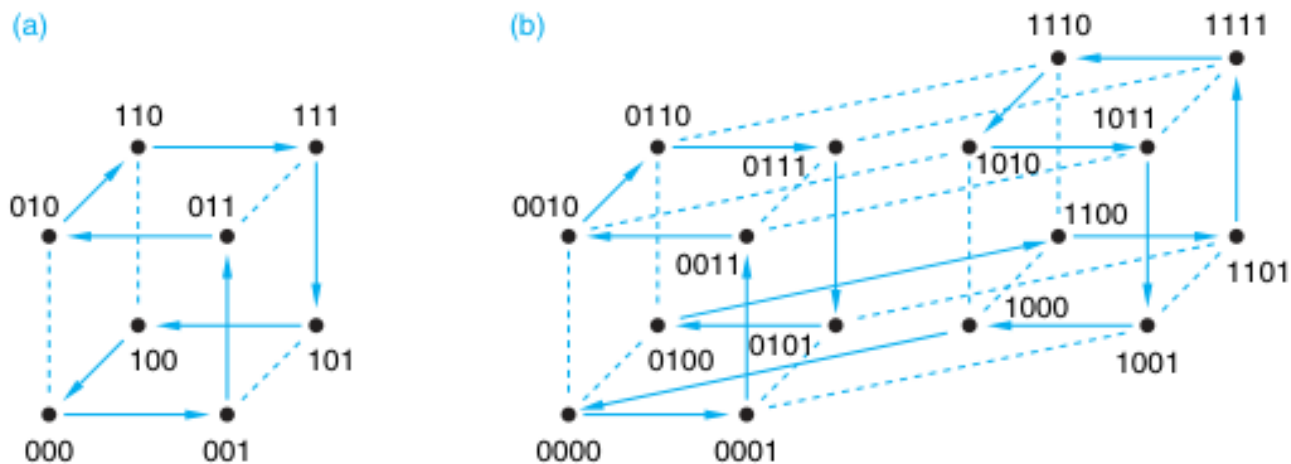
- An n-bit string can be visualized geometrically, as a vertex of an object called an **n-cube**.



Codes

n-Cubes and Distance

- ❑ Example: The problem of designing an n-bit Gray code is equivalent to finding a path along the edges of an n-cube, a path that visits each vertex exactly once.



- ❑ **Hamming distance:** The distance between two n-bit strings is the number of bit positions in which they differ.
- ❑ Example: Two adjacent vertices have distance 1; vertices 001 and 100 in the 3-cube have distance 2 (2 differences in digits).

Errors

- ❑ In digital systems, codes for detecting and correcting errors are essential because physical failures, such as cosmic rays, alpha particles, or overheating, can corrupt data by flipping bits.
 - Example: Sending the ASCII character 'A' (1000001) through a noisy channel where the 6th bit flips, resulting in 1010001
- ❑ The most common is the **Independent Error Model**, which assumes a single physical failure affects only one bit; multiple bit errors are possible but considered less likely
- ❑ Using of Hamming Distance
 - To detect errors of up to d bits, the minimum Hamming distance between valid code words must be $d+1$
 - To correct errors of up to c bits, the minimum distance must be at least $2c+1$

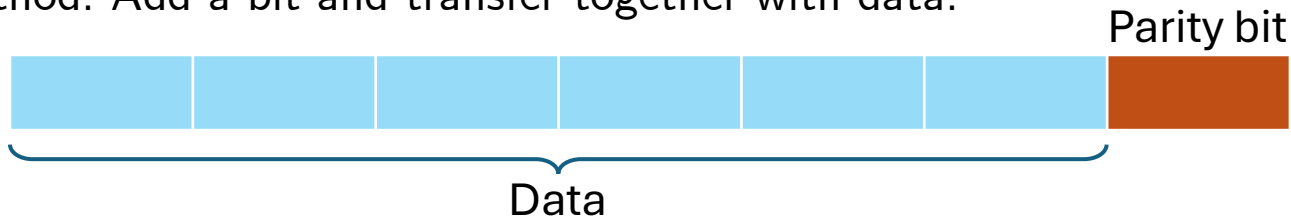
Codes

Errors Detecting Code

❑ These codes notify the system that an error has occurred but generally cannot identify which bit is wrong.

❑ **Parity:** Even and Odd checking.

- Method: Add a bit and transfer together with data.



- Odd parity: The sum of all '1' bit is odd. If the current data has the sum of bit is even, the parity bit is reset to be 0. Else, the parity bit is set to 1.
- Example: Data: 1010001~~x~~. Odd parity. The full string is: 1010010
- Even parity: The sum of all '1' bit is even. If the current data has the sum of bit is even, the parity bit is set to be 1. Else, the parity bit is reset to 0.
- Example: Data: 1010001~~x~~. Even parity. The full string is 1010011

Errors Detecting Code

❑ **Parity:** Even and Odd checking.

- Checking mechanism: The receiver count again the number of '1' bit. The receiver flags an error if it counts an odd number of '1's in the case of even parity and vice versa.
- Limitation: While simple and low-cost, parity has significant weaknesses:
 - *Even Number of Errors:* Parity cannot detect errors if an even number of bits (2, 4, etc.) flip simultaneously, as the total parity remains unchanged
 - *Localization:* It only tells that an error occurred; it cannot identify which bit is incorrect, so it cannot fix the data.

Errors Detecting Code

❑ Checksum

- A checksum is a widely used method for verifying the integrity of a block of data rather than individual words
- At the Sender / Generation Side:
 - The original data is processed through a checksum algorithm to generate a unique, fixed-size checksum value.
 - This checksum value is appended directly to the data packet or stored alongside the file.
- At the Receiver / Verification Side:
 - The receiving device takes the incoming block of data and runs it through the exact same algorithm to compute a new checksum value.
 - Comparison: The newly calculated checksum is compared against the original checksum attached to the data and compare to detect the errors.

Errors Detecting Code

❑ Checksum

- Depending on performance requirements and security needs, different types of checksums are used:
 - Internet Checksum: Based on one's complement addition of 16-bit words. Commonly used in low-layer network protocols like IP, TCP, and UDP for quick error detection.
 - Cyclic Redundancy Check (CRC): Based on binary polynomial division. Highly effective at detecting burst errors in industrial communication (such as Modbus) and storage media (Ethernet, USB, SD cards).
 - Cryptographic Hash Functions (MD5, SHA-1, SHA-256): Advanced cryptographic hashes that act as robust checksums to verify file integrity and protect against malicious tampering.

Errors Detecting Code

❑ Checksum

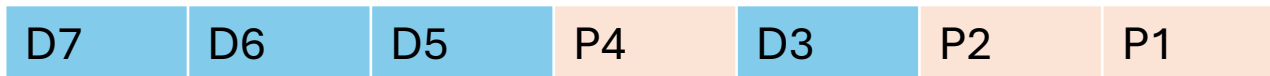
- Limitations
 - No Self-Correction: Like parity bits, standard checksums can only detect errors; they cannot automatically recover or repair the corrupted data (unless error-correcting codes like ECC or Reed-Solomon are applied).
 - Hash Collisions: Simpler checksum algorithms have a finite bit length, meaning two completely different blocks of data could theoretically generate the exact same checksum value..

Codes

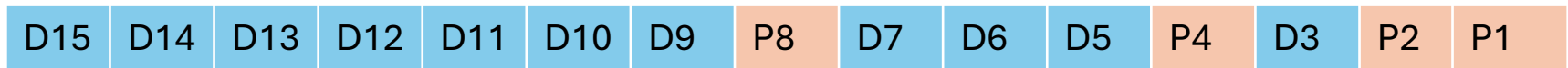
Errors Correcting Code

- ❑ Locates the precise bit index containing the flip and automatically repairs it at the receiver side.
- ❑ Hamming Codes
 - Bit positions are indexed using 1-based numbering from left to right.
 - Positions 2^n (1, 2, 4, 8, 16...) are reserved exclusively for Parity Bits $P_1, P_2, P_4, \dots, P_{2^n}$.
 - All remaining positions (3, 5, 6, 7...) carry Data Bits ($D_3, D_5, D_6, D_7, \dots$).
 - Example:

7-bit code (Hamming (7,4))



15-bit code (Hamming (15,11))



Codes

Errors Correcting Code

❑ Hamming Codes

7-bit code (Hamming (7,4))

D7	D6	D5	P4	D3	P2	P1
----	----	----	----	----	----	----

Position Index	Bit Type	Binary Index	Coverage Membership
1	Parity Bit 1 (P_1)	001	Checks positions with LSB = 1 (1, 3, 5, 7)
2	Parity Bit 2 (P_2)	010	Checks positions with Middle Bit = 1 (2, 3, 6, 7)
3	Data Bit 1 (D_3)	011	Monitored by P_1 and P_2
4	Parity Bit 4 (P_4)	100	Checks positions with MSB = 1 (4, 5, 6, 7)
5	Data Bit 5 (D_5)	101	Monitored by P_1 and P_4
6	Data Bit 6 (D_6)	110	Monitored by P_2 and P_4
7	Data Bit 7 (D_7)	111	Monitored by P_1 , P_2 , and P_4

Codes

Errors Correcting Code

❑ Hamming Codes

- ❑ Example. Data bits 1011 must be transmitted. Construct even-parity, 7-bit, Hamming code for this data.

Solution

1	0	1	P4	1	P2	P1
---	---	---	----	---	----	----

P1 checks P1, D3, D5, D7. $D3 + D5 + D7 = 3$. $\rightarrow P1 = 1$. (even parity)

P2 checks P2, D3, D6, D7. $D3 + D6 + D7 = 2$. $\rightarrow P2 = 0$.

P4 checks P4, D5, D6, D7. $D5 + D6 + D7 = 2$. $\rightarrow P4 = 0$.

Codes

Errors Correcting Code

❑ Hamming Codes

- The location of error bit is $(\overline{...P_4P_2P_1})_2$. If the bit parity P_k error, $P_k=1$.
- The corrected data: flip the error bit.
- Example. Assume that the data has been encoded in a 7-bit even-parity Hamming code and the number 1011011 is received. Find out the bit(s) in error. What will the corrected code be?

Solution

1	0	1	1	0	1	1
---	---	---	---	---	---	---

Bits 1, 3, 5, and 7 associated with parity bit P_1 contain an error. (even parity)

Bits 2, 3, 6, and 7 associated with parity bit P_2 contain no error.

Bits 4, 5, 6, and 7 associated with parity bit P_3 contain an error.

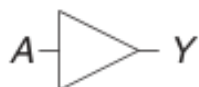
The location is $101_2 = 5_{10}$ (bit D5).

The corrected data is: 1001011.

Logic Gates

Logic Gates and Truth Tables

BUF



$$Y = A$$

A	Y
0	0
1	1

AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR



$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1